

IMPERIAL

IMPERIAL COLLEGE LONDON

DEPARTMENT OF MATHEMATICS

SECOND-YEAR GROUP RESEARCH PROJECT

Dimensionality Reduction

Author:

Akshat Chandel (CID: 02212811)

George Gui (CID: 02426792)

Benjy Menachemson (CID:
02232104)

Leo Rogers (CID: 02391392)

Xuanyu Shi (CID: 02403446)

Supervisor:

Dr. Andrew Duncan

June 16, 2025

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Dimensionality Reduction	3
1.3	Summary of Methods	4
1.4	Project Aim	4
2	Theoretical Framework	5
2.1	Principal Component Analysis	5
2.1.1	Brief Example	5
2.1.2	Method	7
2.1.3	Singular Value Decomposition	9
2.2	Kernel PCA	10
2.3	Autoencoders	11
2.3.1	Neural Networks	11
2.3.2	Linear Autoencoders and their Relationship to PCA	13
2.3.3	Nonlinear Autoencoders	14
2.3.4	Design of Neural Network	15
2.4	Local Linear Embedding	16
2.4.1	Mathematical Foundation	16
2.4.2	LLE Algorithm	17
2.4.3	Brief Example	18
2.5	Comparative Analysis	19
3	Application in Music Genre Classification	20
3.1	Feature Extraction	21
3.2	PCA	21
3.3	Kernel PCA	22
3.4	Autoencoders	23
3.5	LLE	25
4	Conclusion	27
4.1	Theoretical Overview	27
4.2	Empirical Findings	28
4.3	Comparative Insights	28
4.4	Limitations and Future Work	29
4.5	Final Remarks	29

Acknowledgement	29
A Github Repository	30
B Proof of Theorem 2.1	30

1 Introduction

1.1 Motivation

Modern datasets often involve observations embedded in high-dimensional spaces. In the case of audio and music data, even a single song clip can be represented by hundreds of features, such as tempo-related features and spectral contrast. While such representations are necessary to capture the complexity of music, high dimensionality introduces quite a lot of challenges.

From a computational standpoint, machine learning algorithms become more expensive and less reliable as the number of features grows, a phenomenon known as the curse of dimensionality (GeeksforGeeks, 2024). In high dimensions, distances between points become less meaningful, and many algorithms suffer from overfitting or poor generalisation.

Secondly, not every element of the input is equally informative. High-dimensional data frequently has irrelevant fluctuation or noise. For example, certain spectral characteristics may differ in ways orthogonal to the underlying structure that distinguishes different musical genres or may be highly connected.

These concerns highlight the importance of dimensionality reduction, a method that alters high-dimensional data into lower-dimensional data and preserves the most important structures. This process decreases computational costs and the risk of overfitting while improving understanding of the data.

In this project, we investigate several dimensionality reduction methods and assess their performance in the context of music genre classification, a task where high-dimensional audio features must be mapped to discrete, meaningful labels. The goal is to evaluate how well different techniques uncover the underlying structure of the data and support classification accuracy, while also analysing the theoretical principles behind their design.

1.2 Dimensionality Reduction

Dimensionality reduction is the process of transferring data from a high-dimensional space to a lower-dimensional one while preserving as much of the meaningful structure or information as possible (Murel and Kavlakoglu, 2024). Formally, given data points $x_1, x_2 \in \mathbb{R}^D$, the goal is to find a transformation $f : \mathbb{R}^D \rightarrow \mathbb{R}^d$ where $d < D$, such that the low-dimensional representations $f(x_i)$ preserve the essential geometric or statistical properties of the original data.

Dimensionality reduction methods can be classed into two groups:

- **Linear methods** assume that the data lies approximately on a lower-dimensional linear subspace of the ambient space. They attempt to find a set of directions along which the data varies most significantly.
- **Non-linear methods** relax the assumption of linearity and instead attempt to discover manifold structure in the data, curved, lower-dimensional surfaces embedded within the high-dimensional space. These include manifold learning algorithms like Local Linear Embedding (LLE), which maintain local neighbourhoods, and kernel PCA, which applies the PCA framework in a modified feature space. A potent nonlinear technique in recent years has been the emergence of autoencoders, a type of neural networks taught to recreate their inputs via a compressed hidden layer (Pieter Geelen, 2023).

In classification problems, dimensionality reduction is especially useful. Improved generalisation, less computing complexity, and occasionally higher classification accuracy can

be obtained by projecting data into a lower-dimensional space that maintains class structure. Also, data visualisation can offer intuitive insights regarding model performance and class separability.

In this project, we compare four dimensionality reduction methods (PCA, kernel PCA, LLE, and autoencoders) not only in terms of their theoretical underpinnings, but also with respect to their empirical performance in a genre classification task. Each method offers a different perspective on how structure can be extracted from data, and their comparative analysis provides insight into the trade-offs between linearity, locality, nonlinearity, and parametric modelling.

1.3 Summary of Methods

Four dimensionality reduction strategies are examined in this study, each of which offers a unique method for identifying structure in high-dimensional data.

- **Principal Component Analysis (PCA):** is a linear technique that projects data onto orthogonal directions of maximum variance to reduce dimensionality while preserving global structure.
- **Kernel PCA:** By projecting data onto a high-dimensional feature space using a kernel function, Kernel PCA expands on PCA by allowing linear PCA to capture nonlinear structure in that space (Mahajan, 2023).
- **Autoencoders:** Neural network-based models trained to compress data into a low-dimensional latent space and reconstruct it, capturing complex nonlinear structures (encord.com, 2024).
- **Local Linear Embedding (LLE):** A nonlinear technique that reconstructs each data point as a linear combination of its closest neighbours, preserving the local structure of the data.

These four methods offer complementary perspectives on dimensionality reduction, from simple linear projections to kernel based and manifold-preserving embeddings, to neural models trained to reconstruct structured representations. In our project, we evaluate each method in the context of music genre classification, considering their theoretical assumptions, computational behaviour, and classification performance after dimensionality reduction.

1.4 Project Aim

Examining various dimension reduction strategies in relation to musical genres is the aim of this study. Specifically, we aim to investigate how various linear and nonlinear dimensionality reduction methods affect the performance of genre classifiers trained on high-dimensional audio feature representations, respectively.

Our analysis is predicated on two objectives:

- **Theoretical comparison:** We analyse the mathematical underpinnings of each approach, such as the linearity, locality, and structure preservation presumptions. This means understanding how each approach constructs the reduced feature space and the degree of control it provides over generality, interpretability, and dimensionality.

- **Empirical evaluation:** Using a dataset of genre-labelled music songs, we evaluate the practical performance of different dimensionality reduction techniques. We apply the transformation and calculate the accuracy for each approach. Using 2D projections when necessary, we also look at how effectively the reduced representations visually distinguish between genre classifications.

Our goal is to combine theoretical knowledge with empirical data to identify the most effective approaches and the reasons behind them. The GTZAN dataset, which includes 1,000 audio songs from ten different genres, will be used for our research. Each track will be represented by high-dimensional feature vectors extracted from its audio content.

2 Theoretical Framework

In this section, we will explore the four dimension reduction techniques in detail, presenting their mathematical foundation. We will introduce the methods in increasing nonlinearity order. At the end of this section, we will also give a comparative analysis of the methods mentioned, presenting the advantages and disadvantages of each method in terms of computational costs and effectiveness against certain types of data.

2.1 Principal Component Analysis

A dataset may have thousands or millions of dimensions. Our goal is to represent the data with fewer dimensions whilst still capturing most of the variance of our dataset. We want to find the intrinsic dimension of our data. Many methods exist for this, but principal component analysis (PCA) is one of the oldest and most widely used. We aim at finding new variables, the principal components, that are linear combinations of those in the original data set that successively maximise variance and are uncorrelated with each other. The first principal component captures the most variance in the data, the second principal component captures the most remaining variance while being uncorrelated with the first, and so on.

2.1.1 Brief Example

Suppose we have a data set which we centre to get the following scatter plot

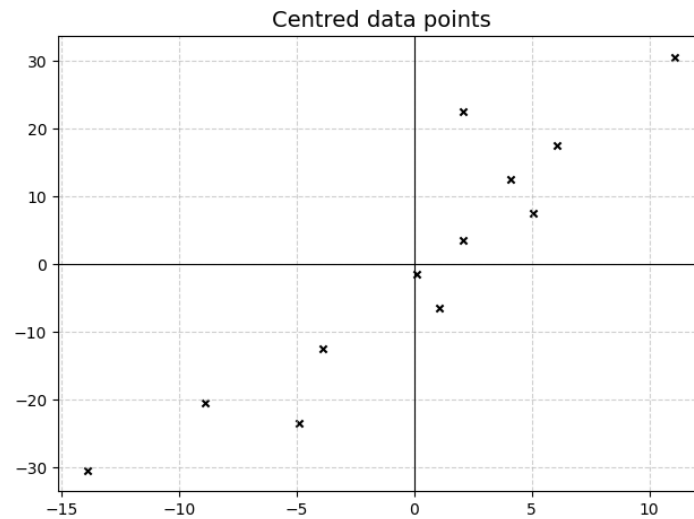


Figure 1

Then we find the principal components that capture most of the variance in our dataset. The method for this will be given below. The first principle component will look like this

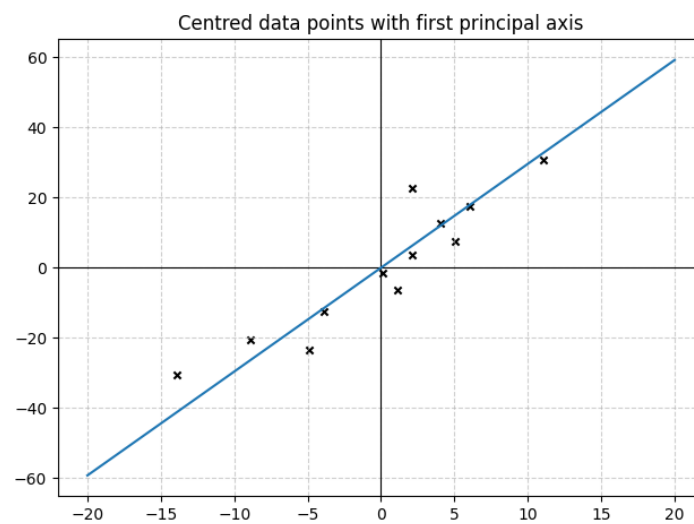


Figure 2

Adding the second principal component to the graph gives

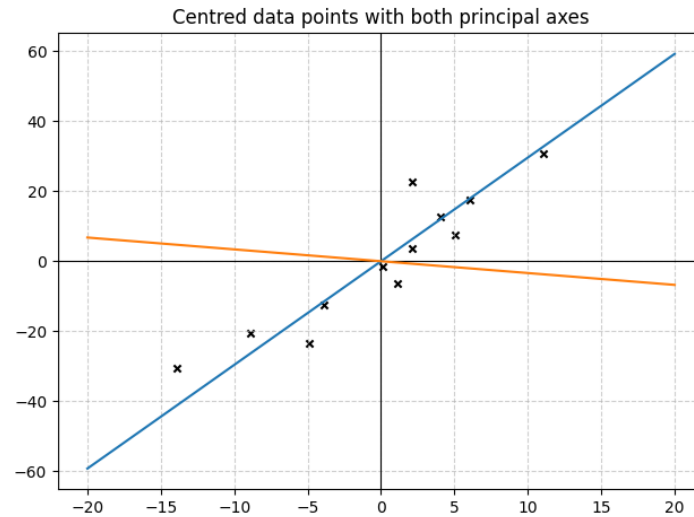


Figure 3

We can plot the variance captured by each principal component and we clearly see the first principal component captures most of the variance of our dataset

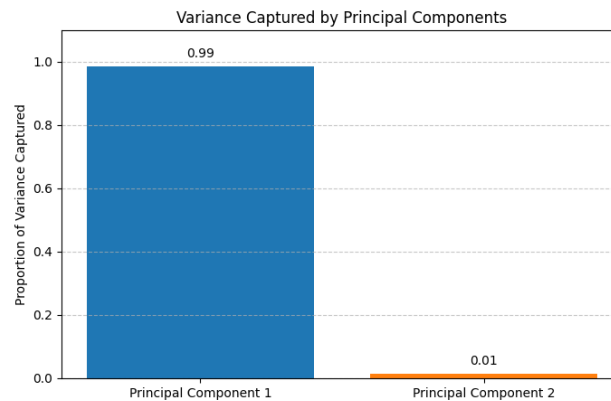


Figure 4

And so we can reduce the dimension of our dataset from 2 to 1 while still capturing 99% of the variance of our dataset. We will now formalise this method below.

2.1.2 Method

Suppose we have a dataset where we have:

- p variables/features
- n observations, $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$, where $\mathbf{x}_i \in \mathbb{R}^p$
- $X \in \mathbb{R}^{n \times p}$ where each row is an observation vector $\mathbf{x}_i$

We aim to find uncorrelated linear combinations of the columns of X that successively have maximum variance. The principal components have the form $X\mathbf{a}_i$, where $\mathbf{a}_i$ is a constant

p -dimensional vector. First we want to find $\mathbf{a}_1$ such that $X\mathbf{a}_1$ has maximum variance. Then we want to find $\mathbf{a}_2$ such that $X\mathbf{a}_2$ is uncorrelated with $X\mathbf{a}_1$, and $X\mathbf{a}_2$ has maximum variance. We continue finding $\mathbf{a}_k$ such that $X\mathbf{a}_k$ is uncorrelated with $X\mathbf{a}_{k-1}$, $X\mathbf{a}_{k-2}$, ..., $X\mathbf{a}_1$ and $X\mathbf{a}_k$ has maximum variance, for $k = 2, \dots, p$.

Usually, we first centre our variables by defining the new matrix X^* where $(X^*)_{ij} = x_{ij} - \bar{x}_j$, where $\bar{x}_j$ denotes the mean value of the j -th feature (i.e. $\bar{x}_j$ is the mean of the j -th column of X). From now on we will refer to X as the centred matrix.

We want to find $\mathbf{a}_1$ such that $\text{Var}(X\mathbf{a}_1)$ is maximised. We have

$$\text{Var}(X\mathbf{a}_1) = \mathbf{a}_1^T \text{Var}(X) \mathbf{a}_1 = \mathbf{a}_1^T S \mathbf{a}_1$$

where S is the sample covariance matrix. Assuming that we are in the large-sample regime, and hence the sample covariance matrix is a good approximation for the population covariance matrix. So we want to maximise $\mathbf{a}_1^T S \mathbf{a}_1$. We need an additional constraint to have a well-defined solution, the most common being to require

$$\mathbf{a}_1^T \mathbf{a}_1 = 1.$$

Maximising $\mathbf{a}_1^T S \mathbf{a}_1$ where $\mathbf{a}_1^T \mathbf{a}_1 = 1$ is equivalent to maximising

$$\mathbf{a}_1^T S \mathbf{a}_1 - \lambda(\mathbf{a}_1^T \mathbf{a}_1 - 1).$$

This is known as the Lagrangian formulation. Differentiating with respect to $\mathbf{a}_1$ and setting equal to $\mathbf{0}$ gives

$$S\mathbf{a}_1 - \lambda\mathbf{a}_1 = \mathbf{0} \iff S\mathbf{a}_1 = \lambda\mathbf{a}_1.$$

So $\mathbf{a}_1$ must be a (unit-norm) eigenvector of S . So choosing $\mathbf{a}_1$ to be a (unit-norm) eigenvector of S maximises $\text{Var}(X\mathbf{a}_1)$. Since $\text{Var}(X\mathbf{a}_1) = \mathbf{a}_1^T S \mathbf{a}_1$, and since now $S\mathbf{a}_1 = \lambda\mathbf{a}_1$, we have

$$\text{Var}(X\mathbf{a}_1) = \mathbf{a}_1^T \lambda \mathbf{a}_1 = \lambda \mathbf{a}_1^T \mathbf{a}_1 = \lambda$$

as $\mathbf{a}_1$ has unit norm. So we want to choose $\mathbf{a}_1$ to be an eigenvector of S with the largest magnitude eigenvalue. Note that since the covariance matrix S is real and symmetric (see below), all its eigenvalues are non-negative, and so the largest magnitude eigenvalue will also be the largest eigenvalue.

S is the sample covariance matrix defined with S_{ij} the sample covariance between x_i and x_j (the i -th and j -th features).

$$S_{ij} = \frac{1}{n-1} \sum_{k=1}^n (x_{ki} - \bar{x}_i)(x_{kj} - \bar{x}_j)$$

where $\mathbf{x}_i = (x_{1i}, x_{2i}, \dots, x_{ni})$, and $\bar{x}_i$ is the mean of variable i . We clearly have $S_{ij} = S_{ji}$, and so S - which is a $p \times p$ matrix - is real and symmetric, and so has p real eigenvalues, whose eigenvectors can be chosen to be orthonormal.

We can show that if $\mathbf{a}_i$ is the i -th eigenvector of S with eigenvalue λ_i , where λ_1 is the largest in magnitude and λ_p is the smallest, then $X\mathbf{a}_i$ successively maximise the variance, subject to being uncorrelated with the previous linear combinations. We can show they are uncorrelated by the following argument: the correlation between $X\mathbf{a}_i$ and $X\mathbf{a}_j$ is given by $\mathbf{a}_j^T S \mathbf{a}_i = \lambda_i \mathbf{a}_j^T \mathbf{a}_i = 0$ if $i \neq j$, since $\mathbf{a}_i$ and $\mathbf{a}_j$ are eigenvectors of S with different eigenvalues.

These linear combinations $X\mathbf{a}_i$ are called the principal components of the dataset.

Reducing the dimension:

Once we have found our principal components, $X\mathbf{a}_1, X\mathbf{a}_2, \dots, X\mathbf{a}_p$, there are a few ways to reduce the dimension of our dataset.

If we have a target intrinsic dimension, say r , we pick the first r principal components; $X\mathbf{a}_1, X\mathbf{a}_2, \dots, X\mathbf{a}_r$, and the original data can be transformed into a lower dimensional (r -dimensional) space using these principal components. This is done by projecting each data point onto these principal component vectors.

If we instead want to reduce the dimension so that we capture α of the variance of our data, we do this by calculating the principal components and use the first k eigenvectors, where k is the smallest number such that $X\mathbf{a}_1, X\mathbf{a}_2, \dots, X\mathbf{a}_k$ captures α of the variance.

Another way to do this is by the elbow method. The elbow method helps determine the optimal number of principal components to keep. We plot a graph as in Figure 4 of the variance explained for each principal component against their number, and the 'elbow' in the curve - where the rate of explained variance slows down - indicates the number of principal components that capture most of the variance.

2.1.3 Singular Value Decomposition

Singular value decomposition (SVD) provides a method for computing PCA. While PCA can be performed through finding the eigenvectors of the covariance matrix, it can also be achieved by applying SVD directly to the (centred) data matrix. In many practical cases, SVD is often preferred due to its numerical accuracy and efficiency.

The singular value decomposition of a real $n \times p$ matrix X is a factorisation of the form

$$X = U\Sigma V^T$$

where

- U is a $n \times r$ matrix with orthogonal columns (the left singular vectors),
- Σ is a $r \times r$ diagonal matrix (the singular values),
- V is an $p \times r$ matrix with orthogonal columns (the right singular values).

Such a decomposition always exists when X is real.

Since our data matrix X is real, we have the SVD $X = U\Sigma V^T$. Therefore we have

$$X^T X = (V\Sigma^T U^T)(U\Sigma V^T) = V\Sigma^2 V^T$$

since $U^T U = I_r$, and Σ is diagonal, so is symmetric.

Thus $X^T X$ is diagonalisable, with eigenvalues given by the squares of the singular values, and eigenvectors given by the columns of V .

The eigenvectors of $X^T X$ are also the eigenvectors of $S = \frac{1}{n-1} X^T X$, and so by computing the SVD of the data matrix X , the columns of V directly give us the principal components. Furthermore, the eigenvalues of S are $\lambda_i = \sigma_i^2 / (n-1)$, where σ_i is the i -th singular value of X . Since the eigenvalues of S represent the variances explained by each principal component, the singular values of X contain this information aswell.

Hence SVD gives us an efficient way to compute both the principal directions - given by the columns of V - and the variances - given by the entries in Σ - without explicitly calculating the covariance matrix.

2.2 Kernel PCA

Principal Component Analysis is a powerful linear technique for dimension reduction, as discussed in the previous section. However, if the dataset has more complicated, non-linear structures which cannot be well represented in a linear subspace, standard PCA will not be very effective. Kernel PCA extends PCA to handle non-linear cases by projecting the dataset into a higher-dimensional feature space, where the structure becomes approximately linear.

Assume we have a non-linear transformation

$$\phi : \mathbb{R}^D \rightarrow \mathbb{R}^M$$

from the original D -dimensional feature space to an M -dimensional feature space, where usually $M \gg D$. So each data point $\mathbf{x}_i \in \mathbb{R}^D$ is projected to a point $\phi(\mathbf{x}_i) \in \mathbb{R}^M$. We can now perform standard PCA in the new feature space, however, this is very inefficient. We can use the kernel methods to perform PCA without ever computing $\phi(\mathbf{x}_i)$ explicitly.

We first assume that the projected new features have zero mean

$$\frac{1}{N} \sum_{i=1}^N \phi(\mathbf{x}_i) = 0.$$

The covariance matrix of the projected features, S_ϕ , is an $M \times M$ matrix, where

$$S_\phi = \frac{1}{N} \sum_{i=1}^N \phi(\mathbf{x}_i) \phi(\mathbf{x}_i)^T.$$

We wish to find eigenvalues λ_k and eigenvectors $\mathbf{v}_k \in \mathbb{R}^M$ such that

$$S_\phi \mathbf{v}_k = \lambda_k \mathbf{v}_k$$

for $k = 1, 2, \dots, M$. Substituting S_ϕ for the form we have above gives

$$\frac{1}{N} \sum_{i=1}^N \phi(\mathbf{x}_i) \{\phi(\mathbf{x}_i)^T \mathbf{v}_k\} = \lambda_k \mathbf{v}_k.$$

Any eigenvector $\mathbf{v}_k$ can be expressed as a linear combination of the mapped data points

$$\mathbf{v}_k = \sum_{j=1}^N a_{kj} \phi(\mathbf{x}_j).$$

Substituting this into the above expression gives

$$\frac{1}{N} \sum_{i=1}^N \phi(\mathbf{x}_i) \{\phi(\mathbf{x}_i)^T \sum_{j=1}^N a_{kj} \phi(\mathbf{x}_j)\} = \frac{1}{N} \sum_{i=1}^N \phi(\mathbf{x}_i) \sum_{j=1}^N a_{kj} \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j) = \lambda_k \sum_{i=1}^N a_{ki} \phi(\mathbf{x}_i).$$

Multiplying both sides on the left by $\phi(\mathbf{x}_l)^T$ gives

$$\frac{1}{N} \phi(\mathbf{x}_l)^T \sum_{i=1}^N \phi(\mathbf{x}_i) \sum_{j=1}^N a_{kj} \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j) = \lambda_k \phi(\mathbf{x}_l)^T \sum_{i=1}^N a_{ki} \phi(\mathbf{x}_i).$$

We now define the kernel matrix $K \in \mathbb{R}^{N \times N}$ with entries

$$K_{ij} = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j) = K(\mathbf{x}_i, \mathbf{x}_j),$$

and the vector $\mathbf{a}_k = (a_{k1}, a_{k2}, \dots, a_{kN})^T$. The above equation now simplifies to

$$\frac{1}{N} \sum_{i=1}^N K_{li} \sum_{j=1}^N a_{kj} K_{ij} = \lambda_k \sum_{i=1}^N a_{ki} K_{li}$$

which we can write as

$$K^2 \mathbf{a}_k = \lambda_k N K \mathbf{a}_k.$$

We can find the eigenvectors $\mathbf{a}_k$ and eigenvalues λ_k by solving the eigenvalue problem

$$K \mathbf{a}_k = \lambda_k N \mathbf{a}_k$$

since K is typically positive semi-definite. To project a new point $\mathbf{x}$ into the k -th principal component, we compute

$$\mathbf{v}_k^T \phi(\mathbf{x}) = \left(\sum_{j=1}^N a_{kj} \phi(\mathbf{x}_j) \right)^T \phi(\mathbf{x}) = \sum_{j=1}^N a_{kj} \phi(\mathbf{x}_j)^T \phi(\mathbf{x}) = \sum_{j=1}^N a_{kj} K(\mathbf{x}_j, \mathbf{x}).$$

And so to project $\mathbf{x}$ into the reduced feature space only involves evaluating the kernel function and avoids any explicit computation in the high-dimensional space.

Common kernel functions include:

- **Linear Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$. This is equivalent to standard PCA.
- **Polynomial Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + c)^d$, where $c > 0$. This maps the dataset into a polynomial with degree d feature space.
- **Gaussian Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / 2\sigma^2)$, which corresponds to mapping into an infinite-dimensional feature space.

2.3 Autoencoders

As the idea of autoencoders is heavily based on neural networks in machine learning, we shall first introduce the basics of neural networks. We will start with the definition of a neural network and introduce the optimisation task that it aims to solve, and then provide an algorithm that efficiently calculates a solution to the optimisation problem.

2.3.1 Neural Networks

A neural network is a mathematical function designed to implement a specific task. It consists of different layers called neurons. A single layer takes in an input vector and applies to it a linear transformation followed by an element-wise non-linear activation function. Formally, a K -layer neural network can be defined as:

$$\mathbf{f}^i = \sigma_i(\mathbf{W}_i \mathbf{f}^{i-1} + \mathbf{b}_i), \quad i = 1, \dots, K,$$

where:

- $\mathbf{f}^i$ denotes the output vector of layer i
- $\mathbf{W}_i$ denotes the weight matrix of layer i
- $\mathbf{b}_i$ denotes the bias term of layer i

- σ_i denotes the activation function of layer i

Note that in the recursive definition above, $\mathbf{f}^0$ is taken to be the input vector $\mathbf{x}$. The weight matrix and the bias term of each layer are parameters to be trained to implement a specific task. The task is commonly encoded in a minimization problem of a loss function. Typically, we choose the loss function to be the Euclidean distance between the output of the neural network and the desired output:

$$\mathcal{L} = \|\hat{\mathbf{x}} - \mathbf{x}\|^2.$$

where we use $\hat{\mathbf{x}}$ and $\mathbf{x}$ to denote the predicted output of the network and the desired output respectively. If we have a dataset of N inputs and desired outputs, the total loss function is defined as:

$$\mathcal{L}_{\text{total}}(\theta) = \sum_{i=1}^N \|\hat{\mathbf{x}}^{(i)} - \mathbf{x}^{(i)}\|^2.$$

where θ denotes a collection of all network parameters, the weights and the biases.

To train and optimise the parameters, we apply **gradient descent**, a method used to iteratively update the parameters based on the input data. On each iteration, we update the parameter by:

$$\theta^{(n+1)} = \theta^{(n)} - \eta \nabla_{\theta} \mathcal{L}_{\text{total}}(\theta^{(n)}),$$

where $\eta > 0$ is called the learning rate and $\nabla_{\theta} \mathcal{L}_{\text{total}}$ is the gradient vector indicating the direction of steepest ascent. We update the parameters in the direction of the steepest descent at each iteration, hence we should arrive near a minimum after sufficient iterations.

Furthermore, to calculate the gradients efficiently, we use an algorithm called **backpropagation**, which utilizes the chain rule to simplify calculations. However, due to time constraints and the focus of this project, we will not delve into the derivation of the method, as long as we grasp the idea of how gradient descent is used to train the parameters. We simply take note that the machine learning libraries implemented in Python implement backpropagation.

In practice, it is common to use variants of the full-batch gradient descent due to concerns over computational cost of taking the gradient over the total loss function. **Stochastic Gradient Descent (SGD)** takes a single sample from the dataset at random each time and calculate the gradient of the loss function with respect to it to update the weight matrix and the bias term. However, this methods suffers from the variance of the samples and may be unstable. A compromise between the two methods, called **Mini-Batch Stochastic Gradient Descent**, takes randomly chosen n samples each time to perform gradient descent. Depending on the size of the mini-batch n , we can find a balance between computational costs and the stability of the model and also optimize the usage of GPUs for training. Therefore, Mini-Batch SGD is usually preferred in practice. Another widely used optimisation algorithm is the **ADAM algorithm** (Adaptive Moment Estimation). It has an adaptive learning rate during the training process and is generally more efficient and more stable. However, in this project we will be focusing on mini-batch SGD and not introduce the ADAM algorithm in detail.

Neural networks can act as classifiers. This is introduced in detail in Chapter 3 of *Mathematical Engineering of Deep Learning*. A common choice of neural networks used to predict the label of data is **Softmax Regression**. The loss function in this case is called Cross-Entropy Loss. We will implement this as a quantifiable test for the quality of dimension reduction of autoencoders.

Autoencoders as Neural Networks

An autoencoder is a neural network used to learn a low-dimensional representation of the data, which preserves as much information as possible and then reconstruct the data so that the difference between the original data and the reconstructed data is minimized. Given the high computational costs of operating on the high-dimensional original data, we may choose instead to operate on the latent representation and then convert the result back to the original dimensions should we need to. Formally, an autoencoder consists of the following two components:

- **Encoder:** A function $\Phi : \mathbb{R}^n \rightarrow \mathbb{R}^m$ that maps input data $\mathbf{x} \in \mathbb{R}^n$ to a latent representation $\mathbf{z} \in \mathbb{R}^m$, where $m < n$.
- **Decoder:** A function $\Psi : \mathbb{R}^m \rightarrow \mathbb{R}^n$ that maps $\mathbf{z}$ back to a reconstruction $\hat{\mathbf{x}} \in \mathbb{R}^n$.

The encoder and decoder functions are trained to minimize a loss function. Typically, this is chosen to be the squared Euclidean distance between the input data $\mathbf{x}$ and the reconstructed data $\hat{\mathbf{x}} = \Psi(\Phi(\mathbf{x}))$:

$$\mathcal{L}(\mathbf{x}, \hat{\mathbf{x}}) = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$$

In practice, the activation functions of neural networks for the encoder and decoder are chosen to be nonlinear functions. However, in the special case where the activation function is chosen as a linear function (e.g. the identity map), the resulting model is known as a linear autoencoder.

2.3.2 Linear Autoencoders and their Relationship to PCA

Taking the activation functions for both the encoder and the decoder to be the identity map, the encoder and decoder functions take the form:

$$\Phi(\mathbf{x}) = \mathbf{W}_e \mathbf{x} + \mathbf{b}_e, \quad \Psi(\mathbf{z}) = \mathbf{W}_d \mathbf{z} + \mathbf{b}_d$$

and therefore the reconstructed data becomes:

$$\hat{\mathbf{x}} = \Psi(\Phi(\mathbf{x})) = \mathbf{W}_d \mathbf{W}_e \mathbf{x} + \mathbf{W}_d \mathbf{b}_e + \mathbf{b}_d.$$

Let $\mathbf{x}^{(i)} \in \mathbb{R}^n$ denote the i -th data point in the dataset, where $i = 1, \dots, N$. Therefore, the optimisation problem involves minimising the reconstruction loss, given by the following expression:

$$\min_{\mathbf{W}_e, \mathbf{b}_e, \mathbf{W}_d, \mathbf{b}_d} \sum_{i=1}^N \left\| \mathbf{x}^{(i)} - \Psi(\Phi(\mathbf{x}^{(i)})) \right\|^2.$$

By substituting the functions for the encoder and decoder, the optimisation problem becomes:

$$\min_{\mathbf{W}_e, \mathbf{b}_e, \mathbf{W}_d, \mathbf{b}_d} \sum_{i=1}^N \left\| \mathbf{x}^{(i)} - \mathbf{W}_d \mathbf{W}_e \mathbf{x}^{(i)} - (\mathbf{W}_d \mathbf{b}_e + \mathbf{b}_d) \right\|^2$$

We will utilize the following theorem to simplify the optimisation problem. Its proof is provided in Appendix B.

Theorem 2.1. Let $\mathbf{W}_e$, $\mathbf{b}_e$, $\mathbf{W}_d$ and $\mathbf{b}_d$ be defined as in the above optimisation problem, and let $\bar{\mathbf{x}} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}^{(i)}$ be the empirical mean of the data. Then:

$$\sum_{i=1}^N \left\| \mathbf{x}^{(i)} - \mathbf{W}_d \mathbf{W}_e \mathbf{x}^{(i)} - (\mathbf{W}_d \mathbf{b}_e + \mathbf{b}_d) \right\|^2 \geq \sum_{i=1}^N \left\| \mathbf{x}^{(i)} - \mathbf{P} \mathbf{x}^{(i)} - (\bar{\mathbf{x}} - \mathbf{P} \bar{\mathbf{x}}) \right\|^2,$$

where $\mathbf{P}$ is the orthogonal projection matrix onto the column space of $\mathbf{W}_d \mathbf{W}_e$.

Equality holds if and only if both of the following conditions are satisfied:

$$\begin{aligned} (\mathbf{P} - \mathbf{W}_d \mathbf{W}_e)(\mathbf{x}^{(i)} - \bar{\mathbf{x}}) &= \mathbf{0}, \quad \forall i \in \{1, \dots, N\}, \\ \mathbf{W}_d \mathbf{b}_e + \mathbf{b}_d &= \bar{\mathbf{x}} - \mathbf{W}_d \mathbf{W}_e \bar{\mathbf{x}}. \end{aligned}$$

The above theorem states that the reconstruction loss of linear autoencoders is minimized when $\mathbf{W}_d \mathbf{W}_e$ performs the same action as the projection matrix $\mathbf{P}$ on the mean-centered data, and that the bias terms match the difference between the data mean and its projection.

Notice now that we are dealing with an optimisation problem on the mean-centered data, analogous to subtracting the mean of features and obtaining a centered matrix in the derivation of PCA. Furthermore, we are now finding a projection matrix that minimizes the reconstruction loss of projection. This is equivalent to the optimisation problem that PCA solves, therefore, $\mathbf{P}$ must be the projection matrix onto the principal subspace, hence the column space of $\mathbf{W}_d$ must be spanned by the first m principal components of the centered dataset.

Thus, trained linear autoencoders project the input data onto the low-dimensional principal subspace. However, without the restriction that the columns of $\mathbf{W}_d$ be orthonormal, they can be correlated and need not be sorted in descending captured variance. As a result, the solutions for reduction to different dimensions are not nested.

2.3.3 Nonlinear Autoencoders

We now generalise beyond the linear case and consider using nonlinear activation functions for the encoder and the decoder. They now take the form:

$$\Phi(\mathbf{x}) = \sigma(\mathbf{W}_e \mathbf{x} + \mathbf{b}_e), \quad \Psi(\mathbf{z}) = \sigma'(\mathbf{W}_d \mathbf{z} + \mathbf{b}_d),$$

where σ and σ' are element-wise nonlinear activation functions. In machine learning, a widely accepted choice for the activation function is the **Rectified Linear Unit (ReLU)**. It is defined as the following:

$$\sigma(x) = \max(0, x).$$

ReLU maps all negative inputs to zero and all positive inputs to itself. This is proven to be a very effective nonlinear activation function while maintaining computational simplicity.

In our exploration, we consider an encoder and decoder each with three layers of the above form. The final layer of both the encoder and the decoder does not apply the activation function (ReLU) to allow negative outputs. By using mini-batch SGD to train the neural network, we obtain expressions for the weights and the biases, which meaningfully compress the data into a latent space, and then reconstruct it while minimising loss. The resulting model is called an **unsupervised autoencoder**, where the training is solely aimed at reducing the reconstruction loss.

Specifically for a classification task, we can also use another model called a **supervised autoencoder**. The total loss for a supervised autoencoder combines the reconstruction loss with the classification loss using a parameter $\lambda > 0$ to weigh the importance of the two objectives:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{reconstruction}} + \lambda \mathcal{L}_{\text{CE}},$$

where $\mathcal{L}_{\text{reconstruction}}$ is the squared Euclidean distance between input and reconstruction, and $\mathcal{L}_{\text{CE}}$ is the cross-entropy loss from softmax regression applied on the latent space. The resulting model will be aimed to not only minimise reconstruction loss but also improve classification accuracy.

2.3.4 Design of Neural Network

Unlike the other methods introduced in this report, there are more practical concerns when implementing neural networks in Python. For instance, in our example of a three-layer encoder and decoder, determining the suitable intermediate dimensions for the data. In practice, we should conduct a neural network architecture search which helps in finding effective architectures for the task, however, due to the time constraints of this project, we will only do preliminary optimisation of our neural network architecture. This section will address some issues that we may encounter before actually implementing neural network training.

Normalization

Normalization is important in neural networks. There are different normalization methods and we would like to consider Global Normalization, Batch Normalization, Layer Normalization, Instance Normalization, and Group Normalization. All of them are equivalent to a linear layer in neural networks, so the structures of the neural networks do not change, and their usage is that with them we train the coefficients of the weight matrices and the bias vectors faster and better, however, due to a prevalent issue in machine learning, it can be difficult to explain.

Considering that we have N samples, each of which is a vector of dimension n . We define x as the sample tensor. $x \in \mathbb{R}^{N \times n}$. x_i is the i -th sample, so $\forall i \in \{1, \dots, N\}$, $x_i \in \mathbb{R}^n$. $x_{i,j}$ is the j -th component of the i -th sample, so $\forall i \in \{1, \dots, N\}$, $\forall j \in \{1, \dots, n\}$, $x_{i,j} \in \mathbb{R}$. All of these normalizations f are functions $\mathbb{R}^{N \times n} \rightarrow \mathbb{R}^{N \times n}$, such that

$$\forall i \in \{1, \dots, N\}, \forall j \in \{1, \dots, n\}, f(x)_{i,j} = \frac{1}{\sqrt{v(x)_{i,j}}} (x_{i,j} e(x)_{i,j}) \gamma + \beta$$

, where e and v are the functions $\mathbb{R}^{N \times n} \rightarrow \mathbb{R}^{N \times n}$ and γ and β are learnt by the data.

For global normalization,

$$\forall i \in \{1, \dots, N\}, \forall j \in \{1, \dots, n\}, e(x)_{i,j} = E(x_{\{1, \dots, N\}, j}) \text{ and } v(x)_{i,j} = \text{Var}(x_{\{1, \dots, N\}, j})$$

. Global normalization is the normalization of samples. When we test some data, we normalize them by the value of the trained data. If the test data is not uniformly obtained, it will not be of much use.

For Batch normalization, the functions e and v are the same as global normalization, but we normalize the data in batches instead of considering the entire sample. The theoretical justification for the effectiveness of Batch normalization is presented by Ioffe and Szegedy (2015).

For Layer Normalization,

$$\forall i \in \{1, \dots, N\}, \forall j \in \{1, \dots, n\}, e(x)_{i,j} = E(x_{i, \{1, \dots, n\}}) \text{ and } v(x)_{i,j} = \text{Var}(x_{i, \{1, \dots, n\}})$$

Its effectiveness is justified by Ba et al. (2016). Layer normalization is the normalization of components in each sample. However, if the values in the sample tensor are different physical quantities, then their quotients are different under different units, so their layer normalizations are different. We prefer to use layer normalizations when this is not the case.

Structure of Linear Layers

We need to design the number of the linear layers and the dimensions of each of them.

The number of linear layers is determined by the number of samples. If the number of linear layers is too large, then the neural network may overfit.

Considering that we expect a neural network function to be injective on the set of sample vectors, which is in other words, that we expect that the outputs of the neural network on different sample vectors are different. Neural networks are compositions of linear functions and activation functions. Therefore, we expect the first function to be injective on the set of sample vectors and each of the other functions to be injective on the range of the previous function. Therefore, if the dimension of the first hidden space is smaller than the dimension of the sample vectors, then the neural network will be at most as good as the one which is the same except the first linear function is replaced by the PCA.

Moreover, the activation function may not be injective on the entire space. Intuitively, to design the function of a neural network that is injective on the sample vectors, we would like to design the dimensions of the hidden spaces to be non-decreasing, however, it is difficult to tell which architecture is better in practice.

In autoencoders, we expect the encoder function and the decoder function to be injective on the set of sample vectors. These two functions are usually designed so that the dimensions of hidden spaces are the same except in reverse order. Therefore, we design the dimensions of the hidden spaces of the encoder and decoder to both be non-decreasing then non-increasing, mirroring each other.

2.4 Local Linear Embedding

Local Linear Embedding (LLE) is a dimension reduction technique which focuses on reducing the dimension of high-dimensional data whilst preserving the underlying geometric structure. As opposed to linear methods like PCA which aim to capture global variances, LLE focuses on approximating local relationships between data points. We assume the data lies on a smooth, low-dimensional manifold within the higher-dimensional space, LLE reconstructs each data point as a weighted combination of its nearest neighbours and uses these locally approximated weights to map the data into a low-dimensional space. This makes LLE effective for visualising and analysing data with non-linear structures.

2.4.1 Mathematical Foundation

The fundamental philosophy behind LLE rests on the assumption that complex, high-dimensional data often lies on or near a smooth, lower-dimensional manifold embedded within the high-dimensional space. This geometric structure enables non-linear dimensionality reduction by approximating local neighbourhood relations through linear patches and then globally optimising their low-dimensional arrangement.

Assumption 2.2 (Manifold Tangent Space Approximation). *Let $\mathcal{M} \subset \mathbb{R}^D$ be a smooth d -dimensional manifold, and let $\mathbf{x} \in \mathcal{M}$. Then, there exists a neighbourhood $\mathcal{U} \subset \mathcal{M}$ of $\mathbf{x}$ such*

that for all $\mathbf{x}' \in \mathcal{U}$, the following approximation holds:

$$\mathbf{x}' = \mathbf{x} + \sum_{k=1}^d \alpha_k \mathbf{v}_k + o(\|\mathbf{x}' - \mathbf{x}\|),$$

where:

- $\{\mathbf{v}_1, \dots, \mathbf{v}_d\}$ is an orthonormal basis for the tangent space $T_{\mathbf{x}}\mathcal{M}$,
- $\alpha_k \in \mathbb{R}$ are coefficients encoding the local coordinates of $\mathbf{x}'$ in $T_{\mathbf{x}}\mathcal{M}$,
- $o(\|\mathbf{x}' - \mathbf{x}\|)$ denotes higher-order terms vanishing faster than $\|\mathbf{x}' - \mathbf{x}\| \rightarrow 0$.

This formalises the geometric idea that the tangent space of the manifold on which the data lies can be locally approximated as a hyperplane "just touching" the manifold. This implies $\mathbf{x}_i$ should be reconstructable from its neighbours using weights W_{ij} that are invariant to global rotations, translations, and scalings of the dataset.

Given a dataset $\mathbf{X} = \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N \subset \mathbb{R}^D$ consisting of N data points in a D -dimensional space, LLE aims to discover a corresponding low-dimensional representation $\mathbf{Y} = \mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N \subset \mathbb{R}^d$, where the target dimension d is significantly smaller than the original dimension D (i.e., $d \ll D$). The Johnson-Lindenstrauss lemma states it is possible for high dimensional data to be embedded into much lower dimensions whilst nearly preserving distances between points.

Theorem 2.3 (Johnson-Lindenstrauss Lemma). *Let $0 < \varepsilon < 1$ be given, and let $X \subset \mathbb{R}^n$ be a set of N points. For any integer k satisfying*

$$k > \frac{8 \ln N}{\varepsilon^2},$$

there exists a linear map $f: \mathbb{R}^n \rightarrow \mathbb{R}^k$ such that for all $u, v \in X$,

$$(1 - \varepsilon)\|u - v\|^2 \leq \|f(u) - f(v)\|^2 \leq (1 + \varepsilon)\|u - v\|^2.$$

The proof of Theorem 2.3 relies on advanced techniques from high-dimensional geometry and is beyond the scope of this report. Interested readers may refer to (Johnson-Lindenstrauss, 1984) for a complete derivation.

2.4.2 LLE Algorithm

The LLE algorithm's core idea is its two stage approach to manifold learning. First, LLE operates on the principle that each data point $\mathbf{x}_i$ can be accurately reconstructed as a weighted linear combination of its k nearest neighbours. The local reconstruction relationship is expressed mathematically as

$$\mathbf{x}_i \approx \sum_{j \in \mathcal{N}k(i)} w_{ij} \mathbf{x}_j, \quad \text{subject to} \quad \sum_j w_{ij} = 1,$$

where $\mathcal{N}k(i)$ represents the set of k nearest neighbours of point $\mathbf{x}_i$, and w_{ij} are the optimal reconstruction weights. k is a hyperparameter which can be varied, lower values of k will create higher sensitivity to noise whilst higher values will increase over smoothing. These weights are constrained to sum to unity ensuring they form a proper convex combination that captures the

local geometric relationships within each neighbourhood. The weights $\{w_{ij}\}$ are obtained by minimizing the local reconstruction error

$$\min_{\{w_{ij}\}} \sum_{i=1}^N \left\| \mathbf{x}_i - \sum_{j \in \mathcal{N}_i} w_{ij} \mathbf{x}_j \right\|^2.$$

The weights are computed by solving a local Gram-matrix system with sum-to-one constraints. For a detailed derivation of the weight computation, we refer to [Ghojogh et al., 2020] (see Section 2.2).

The local linear relationships captured by these reconstruction weights should remain invariant under the non-linear transformation to the lower-dimensional embedding space. In the second stage the algorithm optimises lower dimensional coordinates $\mathbf{y}_i$ by minimising the reconstruction error

$$\min_{\mathbf{Y}} \sum_{i=1}^N \left\| \mathbf{y}_i - \sum_{j \in \mathcal{N}_i} w_{ij} \mathbf{y}_j \right\|^2,$$

while keeping the computed weights w_{ij} fixed. The second optimisation finds an embedding in which the relationships in the high-dimensional space are carried into a lower dimension, "unfolding" the underlying manifold structure while maintaining as accurately the local geometric structure of the data.

2.4.3 Brief Example

An easily visualised example of LLE is the Swiss Roll dataset, a 3D manifold where points spiral outwards staying on a 2D surface. The nonlinear Swiss Roll is ideal for illustrating the ability of LLE to preserve local relationships through a dimensionality reduction. Our embedded image (Fig 6) shows the LLE unrolling of the Swiss Roll into a 2D plane by simply maintaining relationships between nearby points, successfully recovering the intrinsic 2D geometry obscured in the original 3D representation.

Swiss Roll in Ambient Space

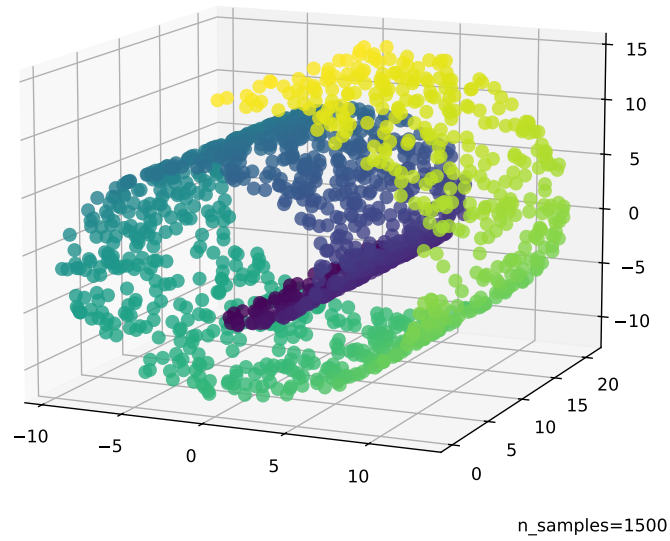


Figure 5 3D Swiss Roll Dataset

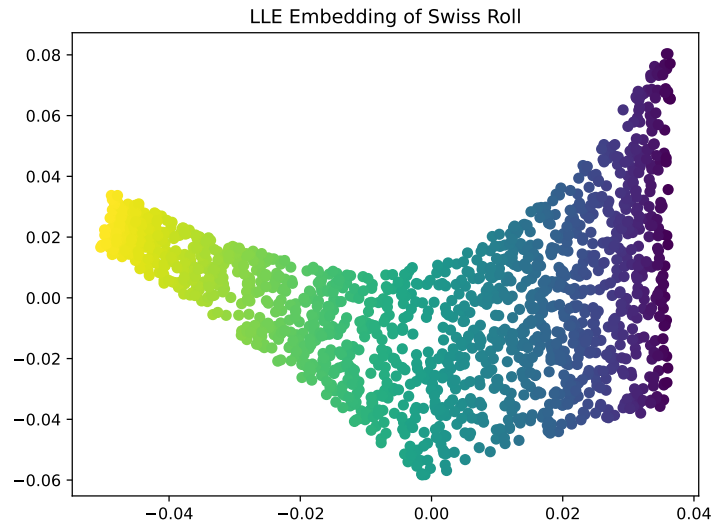


Figure 6 2D LLE Embedding of Swiss Roll

2.5 Comparative Analysis

Having introduced the four different methods for dimensionality reduction, namely PCA, Kernel PCA, Autoencoders and LLE, we will now discuss the advantages and disadvantages of each method from a theoretical standpoint. The comparative analysis will focus on the computational cost, mathematical assumptions, and representational accuracy of each method.

Principal Component Analysis (PCA)

- **Advantages:** PCA is usually computationally simple, as we can apply Singular Value Decomposition to the data matrix and readily obtain the eigenvectors and eigenvalues. Furthermore, the results are easily interpretable as the principal components are the directions which preserve maximum variance.
- **Disadvantages:** PCA relies on the assumption that the data lies near a linear subspace. Otherwise, the mean squared error will be too large for the projection to be meaningful. Moreover, it cannot capture nonlinear relationships within the data.

Kernel PCA

- **Advantages:** Kernel PCA extends PCA to a nonlinear setting, and can therefore capture nonlinear relationships within the data. It is also computationally efficient as it involves eigenvalue decomposition, similar to PCA.
- **Disadvantages:** Kernel PCA heavily relies on the choice of higher-dimensional mapping, which in most cases is nontrivial. The resulting components are less interpretable than those of PCA, since they have embedded the nonlinear structure of the feature space.

Autoencoders

- **Advantages:** Autoencoders utilize neural networks to empower the learning of complex nonlinear structures of the data. By using mini-batch SGD, the training process remains computationally efficient even when dealing with large datasets. The model can be adjusted to a supervised one to incorporate the goal of other tasks.
- **Disadvantages:** The training process may involve solving a non-convex optimisation problem, in which the parameters converge to a local minimum instead of the global minimum. More complex training also requires specialised computational resources. The learned parameters are more difficult to interpret compared to the previous methods. It also involves carefully chosen hyperparameters and strategically constructed neural network architecture.

Local Linear Embedding (LLE)

- **Advantages:** LLE preserves local structure by ensuring that the data is approximately represented as a weighted combination of its neighbours. The only hyperparameter adjustable is the choice of number of neighbours to consider.
- **Disadvantages:** LLE requires the data to lie on or near a nonlinear subspace. In practice, this requires the collected data to be constrained rather than being purely random. Its computational complexity also increases rapidly with the size of the dataset. Furthermore, it does not provide an explicit mapping to the low-dimensional space for a new sample.

3 Application in Music Genre Classification

In this section, we will focus on applying the previously introduced dimensionality reduction techniques to the GTZAN dataset, a dataset containing one thousand 30-second audio files in .wav form, from 10 different genres. The implementation will be done in Python. We will first

extract the important features of the audio files. We then use the different methods to reduce the dimension of the features data to two dimensions and plot the two-dimensional data to recognize any clustering amongst audio files of the same genre.

3.1 Feature Extraction

For each audio file in the GTZAN dataset, we extract the following six features using the `librosa` library:

- **MFCCs (Mel-Frequency Cepstral Coefficients):** A set of 20 coefficients representing the short-term power spectrum of sound, capturing timbral and spectral characteristics.
- **Chroma (Chroma Short-Time Fourier Transform):** A 12 dimensional vector representing the energy distribution across pitch classes (e.g., C, C#, D), useful for harmonic analysis.
- **Spectral Contrast:** Measures the difference in spectral peak and valley magnitudes across frequency sub-bands, highlighting tonal vs. noisy components.
- **Tonnetz (Tonal Centroid Features):** A 6 dimensional representation of harmonic relationships in the tonal space, based on music theory concepts.
- **Tempo (Beats per Minute, BPM):** A scalar value estimating the global tempo of the audio track, derived from beat tracking.

This results in a 66D feature space which we will project onto 2D.

3.2 PCA

We can take the GTZAN dataset which contains 1,000 audio files of songs from 10 different genres, and we can reduce the dimension of the dataset to 2 dimensions and see clustering of different genres. We applied PCA to the feature set extracted from the GTZAN dataset, and projected the feature vectors onto a two-dimensional subspace defined by the first 2 principal components. The following plots were produced in Python using the `librosa` and `sklearn` packages. We obtain the resulting scatter plot

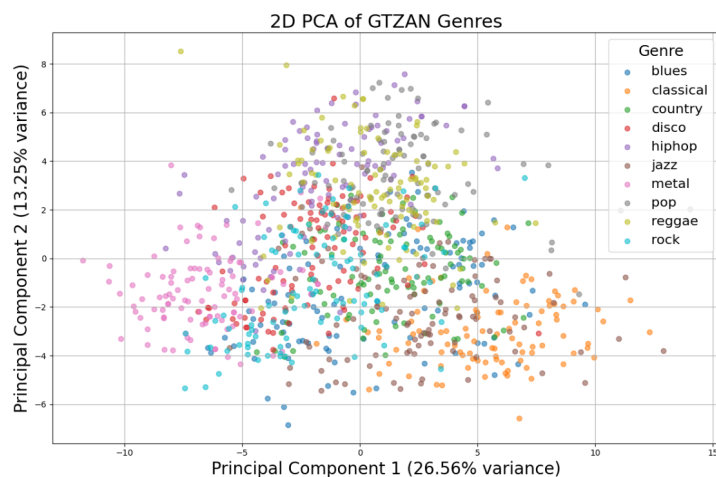


Figure 7 2D GTZAN PCA

which illustrates the distribution of all 10 GTZAN genres. The axes are the first 2 principal components, and the percentages are the proportion of the total variance captured from the original high-dimensional dataset. We can see that while there is some clustering of songs from the same genre, there is still a considerable degree of overlap among the majority of genres, which we would expect as we have used a purely linear dimensionality reduction technique, and we have projected onto a very low dimension that doesn't capture most of the variance. Although PCA does not clearly separate all music genres, if we take a look at only 2 genres, for example Classical and Metal, we can see there is distinct clustering.

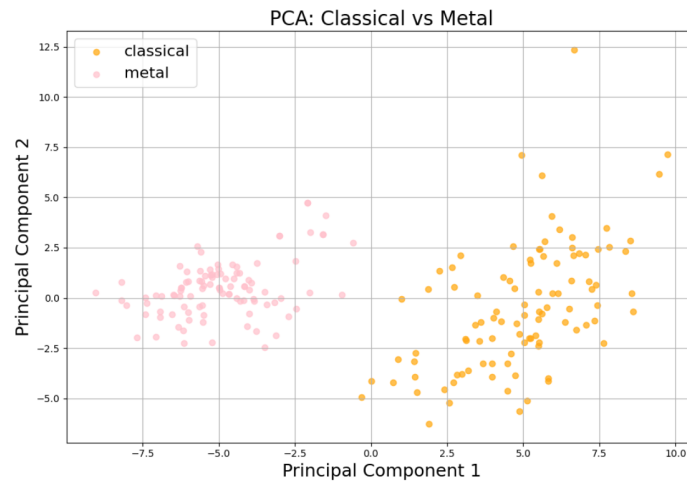


Figure 8 PCA Classical vs. Metal

We can also plot the variance explained by each principal component

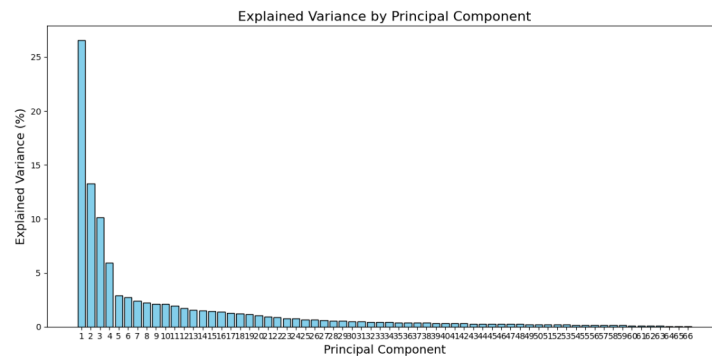


Figure 9 PCA Explained Variance

and we see that the first few principal components capture a majority of the variance in our data set, indicating that PCA does a good job at reducing the dimension of our dataset.

3.3 Kernel PCA

Below we have applied kernel PCA to our dataset using two different kernel functions: a polynomial kernel and a radial basis function (RBF) kernel.

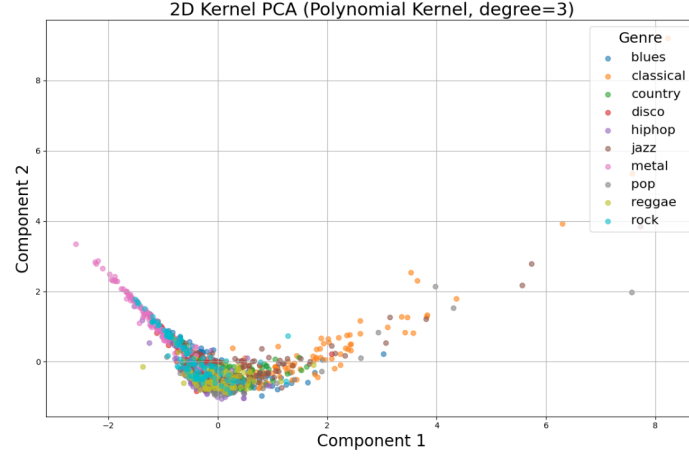


Figure 10 2D GTZAN Polynomial Kernel PCA

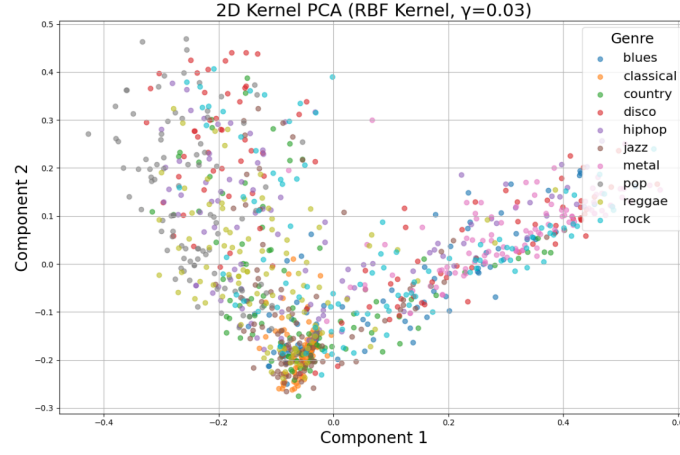


Figure 11 2D GTZAN RBF Kernel PCA

In both cases, the separation between the genres is not captured well, suggesting that neither kernel is suitable for dimensionality reduction for this dataset. This may be because we are reducing to a dimension of 2 which may be too low to capture the behaviour of the dataset.

3.4 Autoencoders

We have 4 neural networks, an autoencoder, a softmax classifier on the transformed 66-dimensional feature space, a softmax classifier on the latent space, and a supervised autoencoder. The reduced dimension is 2. The autoencoder is trained on the data set for 1000 times. The classifiers are trained on the random of the data set for 1000 times and are tested on the other 1/8 of the data set.

In the following, we will adopt the abbreviations: **GN** denotes the global normalization layer, **L(a,b)** refers to a linear layer which takes in an a -dimensional input and gives a b -dimensional output, and **A** represents the activation function, which we take to be ReLU.

The design of the encoder of the supervised and unsupervised autoencoders is as following:

$$\text{GN} \rightarrow \text{L}(66, 132) \rightarrow \text{A} \rightarrow \text{L}(132, 136) \rightarrow \text{A} \rightarrow \text{L}(136, 4) \rightarrow \text{A} \rightarrow \text{L}(4, 2).$$

The design of the decoder of the supervised and unsupervised autoencoders is as following:

$$L(2, 4) \rightarrow A \rightarrow L(4, 136) \rightarrow A \rightarrow L(136, 132) \rightarrow A \rightarrow L(132, 66).$$

The design of the classifier on the feature space is:

$$GN \rightarrow L(66, 68) \rightarrow A \rightarrow L(68, 10).$$

The design of the classifier on the latent space is:

$$GN \rightarrow L(2, 68) \rightarrow A \rightarrow L(68, 10).$$

The design of the classifier in the supervised autoencoder is:

$$L(2, 68) \rightarrow A \rightarrow L(68, 10).$$

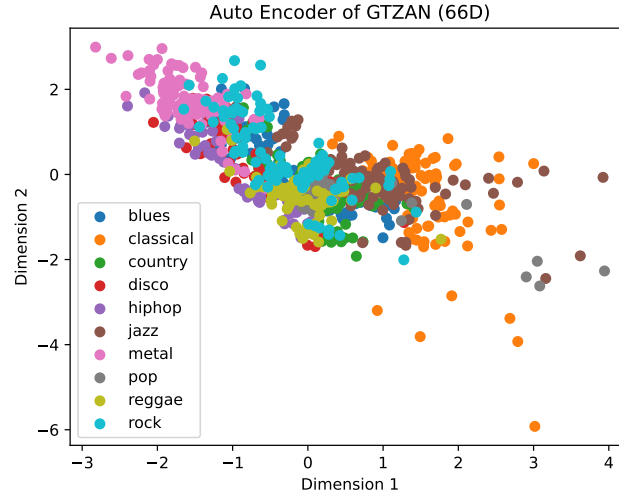


Figure 12 GTZAN Unsupervised Autoencoder

Figure 12 is the 2D plot of the points after reducing the dimensions using the encoder of the unsupervised autoencoder. We can see that the points of different types of music are close to each other. This is rather expected, as the unsupervised autoencoder does not classify the data points according to their genres.

We apply the classifier in the latent space on the test data and achieved an accuracy of 48.8%, compared to an accuracy of 74.4% from the classifier on the feature space.

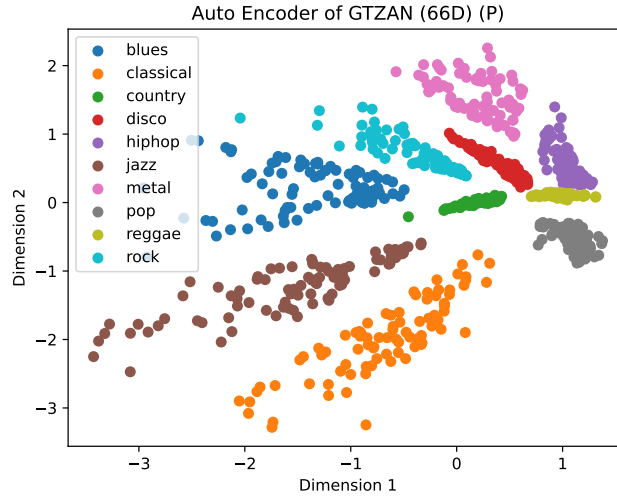


Figure 13 2D GTZAN Supervised Autoencoder (trained data)

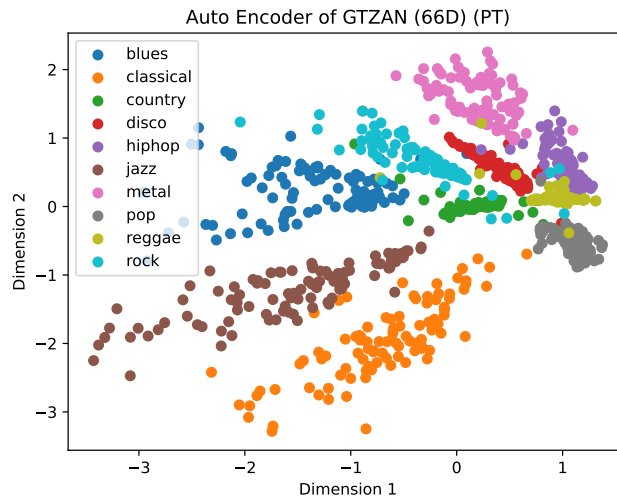


Figure 14 2D GTZAN Supervised Autoencoder (entire data)

Figures 13 and 14 are the 2D plots of the points in the trained data set and the entire data set, after reducing the dimensions using the encoder of the supervised autoencoder. We can see that different music genres are separated clearly, and that for the newly encountered test data, most of them lie close to the clusters of their genres.

The accuracy of the supervised autoencoder, when applied to the test data, is 56%. This is not exceptional, and may be due to the fact that the size of the trained data is small.

3.5 LLE

The LLE embeddings of the GTZAN music genre dataset reveal how varying the neighbourhood parameter k influences sensitivity to noisy data points and becomes oversmoothed as k becomes large.

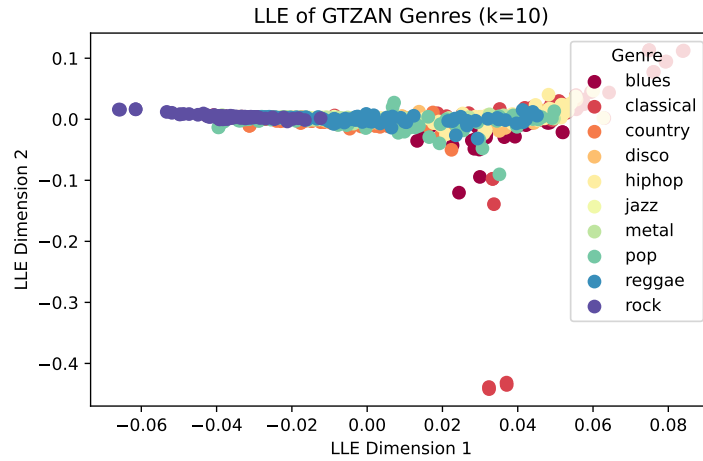


Figure 15 2D GTZAN Embedding Using $k = 10$

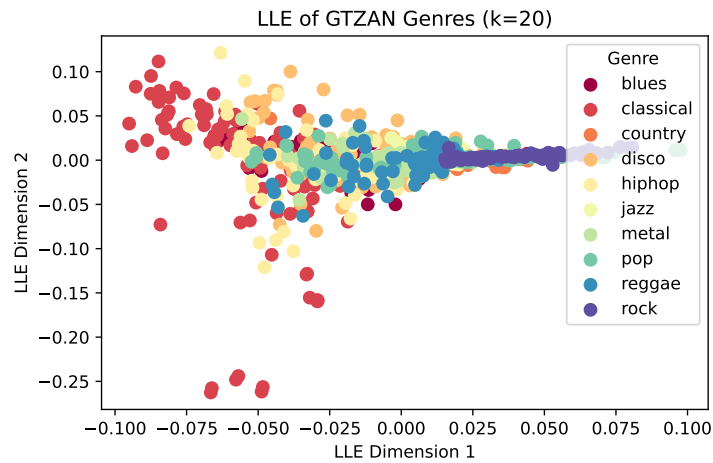


Figure 16 2D GTZAN Embedding Using $k = 20$

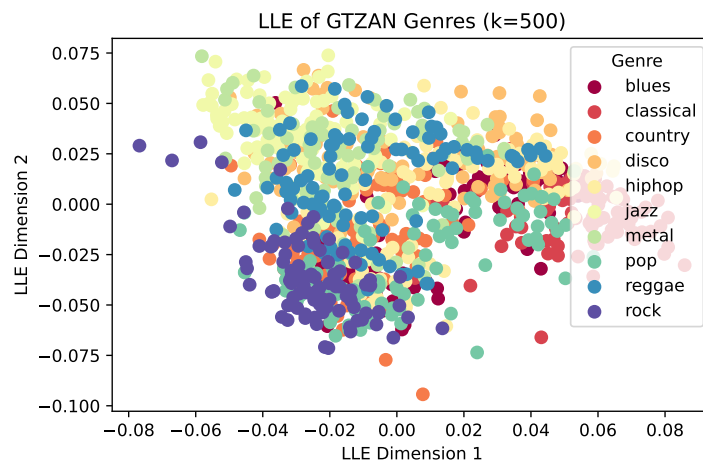


Figure 17 2D GTZAN Embedding Using $k = 500$

Whilst clustering is not clear for all music genres when projecting into 2D in the GTZAN dataset, some such as Rock and Classical exhibit very distinct clustering, indicating that their

audio features are more readily distinguishable in the embedded space and dimension reduction has been successful for these two genres.

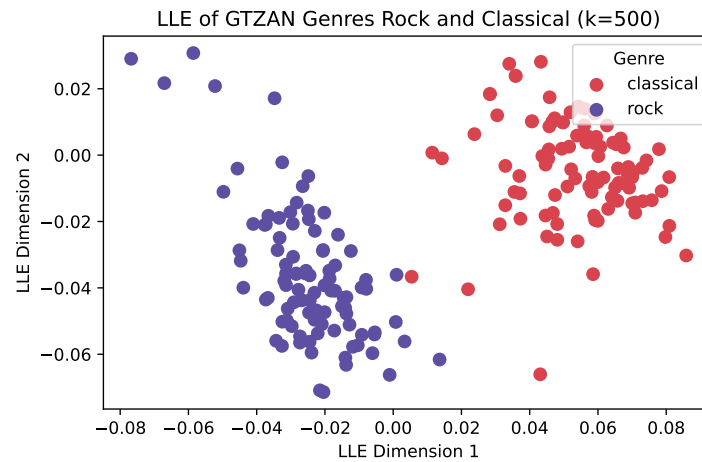


Figure 18 LLE Rock vs. Classical

4 Conclusion

In this project, we investigated four influential dimensionality reduction methods with a focus on both theoretical understanding and empirical performance in music genre classification using the GTZAN dataset. The study aimed to examine how each method uncovers structure in high-dimensional audio features, their assumptions, computational behaviour, and their effectiveness at supporting classification tasks.

4.1 Theoretical Overview

The theoretical analysis provided a clear understanding of the principles that each method follows. By projecting data onto directions of highest global variance under the assumption of linearity, PCA provided a solid foundation. This was extended by kernel PCA, which uses kernel functions to implicitly capture nonlinear structures in a high-dimensional feature space. This is, however, at the expense of careful kernel selection and tuning.

Autoencoders used neural networks to learn nonlinear representations through an encoder-decoder framework, generalising PCA when linear activations are used but offering far greater flexibility with nonlinear activations and carefully designed architectures. LLE, meanwhile, presented a manifold learning approach that preserves local neighbourhood relationships, aligning with the idea that complex data often lies on a smooth, lower-dimensional manifold embedded in a high-dimensional space.

Each method thus balances a trade-off between simplicity, interpretability, and the capacity to capture complex structures. While PCA is transparent and computationally light, Kernel PCA, Autoencoders, and LLE introduce increasing nonlinearity, richer representations, and higher computational demands.

4.2 Empirical Findings

Applied to the GTZAN dataset, the methods showed consistent patterns matching our theoretical expectations.

PCA reduced the 66-dimensional audio feature space to two dimensions for visualisation. The projections demonstrated partial genre clustering; distinct clusters were most visible for genres with markedly different timbral and spectral characteristics, such as Classical and Metal. However, significant overlap persisted for genres with slight acoustic differences, demonstrating PCA's limitation in modelling nonlinear boundaries.

Kernel PCA used polynomial and radial basis function kernels. Interestingly, for the GTZAN dataset and the chosen 2D projections, neither kernel produced clear genre separation compared to PCA. This suggests that while Kernel PCA is theoretically capable of capturing nonlinearity, its performance depends critically on kernel choice and parameter tuning—and a very low target dimension may be too restrictive to reveal the true structure.

Autoencoders demonstrated the most flexible approach. The unsupervised autoencoder learned a compact, nonlinear representation of the audio data but did not separate genres clearly on its own. In contrast, the supervised autoencoder, which combined reconstruction loss with classification loss, worked significantly better: it produced low-dimensional embeddings that clearly separated different music genres in 2D plots and achieved higher test accuracy than the unsupervised version. This shows that adding supervision during training improves the discriminative power of the latent space, although it still requires careful design, normalisation, and regularisation to prevent overfitting.

LLE offered a different viewpoint. We could see how local structure preservation reveals the manifold underlying the audio features by varying the neighbourhood parameter k . Certain genres, such as rock and classical, created unique clusters for moderate k , supporting the theory that local commonalities play a key role in genre characteristics. However, a big k resulted in oversmoothing and the loss of significant clusters, while a low k made embeddings noisy and susceptible to outliers. Furthermore, a drawback of LLE for real-time applications is that it does not produce an explicit mapping for new data points.

4.3 Comparative Insights

Overall, our results reveal that:

- Linear methods are valuable baselines. They're fast and easy to interpret, but they are limited.
- Kernel PCA extends PCA's power but is sensitive to kernel choice and may not perform optimally without careful hyperparameter selection.
- Although autoencoders are flexible and can capture complex correlations, they also add complexity to training and require a large amount of data and processing power. In particular, the supervised autoencoder outperformed the unsupervised version by producing clearer clusters and higher test accuracy, demonstrating the benefit of integrating label information directly into the dimensionality reduction process.
- LLE is effective at revealing local manifold structure but lacks an explicit projection function and scales poorly for large datasets.

These findings show that nonlinear methods are generally more successful with high-dimensional audio data, but they also have higher implementation and processing costs.

The "best" method for reducing dimensionality is not a single one, per se. Instead, the choice is determined by weighing the trade-offs between interpretability, training effort, and dataset type.

4.4 Limitations and Future Work

Several practical limitations shaped this study. Firstly, reducing dimensionality to only two dimensions, while useful for visualisation, may underrepresent the true capacity of nonlinear methods like Kernel PCA and Autoencoders. Exploring embeddings in higher dimensions could provide clearer separation and boost classification accuracy.

Secondly, the GTZAN dataset, although standard, has known issues such as possible mislabelling and limited genre diversity. Future work could use larger, cleaner, and more diverse music datasets to test scalability and generalisability. Future extensions could include:

- Integrating supervised signals in training autoencoders (supervised autoencoders) to directly maximise genre discriminability.
- Experimenting with more advanced manifold learning techniques or deep generative models such as Variational Autoencoders or Generative Adversarial Networks(Lawton, 2023).
- Developing hybrid methods that combine the interpretability of linear projections with the representational power of nonlinear embeddings.
- Optimising methods for real-time streaming scenarios through incremental or online learning algorithms.

4.5 Final Remarks

To sum up, this effort connects the theoretical and practical viewpoints on dimensionality reduction for the classification of musical genres. Through a methodical comparison of PCA, Kernel PCA, Autoencoders, and LLE, we have shown how each approach strikes a compromise between the essential trade-offs of computational feasibility, understanding and simplicity.

Our understanding of the application of dimensionality reduction techniques in music data analysis has improved and the accuracy of music classification systems should be enhanced by further research in this field, enabling more intricate human-computer interactions with large music archives.

Overall, our results highlight that supervised autoencoders were the most effective method tested, achieving the best balance between low-dimensional representation and genre classification performance.

Acknowledgments

We would like to express our sincere gratitude for the help given by our supervisor, Dr. Andrew Duncan, for his guidance, feedback and support throughout the project, and particularly, for helping in resolving issues in the implementation of autoencoders.

A Github Repository

The full code and supplementary materials are available at: <https://github.com/LeoR0gers/M2R.git>

B Proof of Theorem 2.1

We first restate the theorem that we want to prove:

Theorem B.1 (Restated). *Let $\mathbf{W}_e, \mathbf{b}_e, \mathbf{W}_d, \mathbf{b}_d$ be defined as in the reconstruction loss of the linear autoencoder. Then:*

$$\sum_{i=1}^N \left\| \mathbf{x}^{(i)} - \mathbf{W}_d \mathbf{W}_e \mathbf{x}^{(i)} - (\mathbf{W}_d \mathbf{b}_e + \mathbf{b}_d) \right\|^2 \geq \sum_{i=1}^N \left\| \mathbf{x}^{(i)} - \mathbf{P} \mathbf{x}^{(i)} - (\bar{\mathbf{x}} - \mathbf{P} \bar{\mathbf{x}}) \right\|^2,$$

where $\mathbf{P}$ is the orthogonal projection matrix onto the column space of $\mathbf{W}_d \mathbf{W}_e$, and $\bar{\mathbf{x}}$ is the sample mean.

Equality holds if and only if:

$$(\mathbf{P} - \mathbf{W}_d \mathbf{W}_e)(\mathbf{x}^{(i)} - \bar{\mathbf{x}}) = 0, \quad \forall i = 1, \dots, N,$$

$$\mathbf{W}_d \mathbf{b}_e + \mathbf{b}_d = \bar{\mathbf{x}} - \mathbf{W}_d \mathbf{W}_e \bar{\mathbf{x}}.$$

Proof. $\mathbf{P}$ is the orthogonal projection matrix of the column space of $\mathbf{W}_d \mathbf{W}_e$.

Therefore, $I - \mathbf{P}$ is the orthogonal projection matrix of the orthogonal complement of the column space of $\mathbf{W}_d \mathbf{W}_e$.

Therefore,

$$(I - \mathbf{P})^T (\mathbf{P} - \mathbf{W}_d \mathbf{W}_e) = (I - \mathbf{P})(\mathbf{P} - \mathbf{W}_d \mathbf{W}_e) = 0$$

.

Define $a, b, c \in \mathbb{R}^{N \times N}$,

$$\forall i \in \{1, \dots, N\}, a_i = \mathbf{x}^{(i)} - \mathbf{W}_d \mathbf{W}_e \mathbf{x}^{(i)} - (\mathbf{W}_d \mathbf{b}_e + \mathbf{b}_d)$$

$$\forall i \in \{1, \dots, N\}, b_i = \mathbf{x}^{(i)} - \mathbf{P} \mathbf{x}^{(i)} - (\bar{\mathbf{x}} - \mathbf{P} \bar{\mathbf{x}})$$

$$\forall i \in \{1, \dots, N\}, c_i = \mathbf{x}^{(i)} - \mathbf{W}_d \mathbf{W}_e \mathbf{x}^{(i)} - (\bar{\mathbf{x}} - \mathbf{W}_d \mathbf{W}_e \bar{\mathbf{x}})$$

.

Therefore,

$$\sum_{i=1}^N (a_i - c_i)^T c_i$$

$$\begin{aligned}
&= \sum_{i=1}^N ((\bar{\mathbf{x}} - \mathbf{W}_d \mathbf{W}_e \bar{\mathbf{x}}) - (\mathbf{W}_d \mathbf{b}_e + \mathbf{b}_d))^T (\mathbf{x}^{(i)} - \mathbf{W}_d \mathbf{W}_e \mathbf{x}^{(i)} - (\bar{\mathbf{x}} - \mathbf{W}_d \mathbf{W}_e \bar{\mathbf{x}})) \\
&= \sum_{i=1}^N ((\bar{\mathbf{x}} - \mathbf{W}_d \mathbf{W}_e \bar{\mathbf{x}}) - (\mathbf{W}_d \mathbf{b}_e + \mathbf{b}_d))^T (\mathbf{x}^{(i)} - \mathbf{W}_d \mathbf{W}_e \mathbf{x}^{(i)} - (\bar{\mathbf{x}} - \mathbf{W}_d \mathbf{W}_e \bar{\mathbf{x}})) \\
&= ((\bar{\mathbf{x}} - \mathbf{W}_d \mathbf{W}_e \bar{\mathbf{x}}) - (\mathbf{W}_d \mathbf{b}_e + \mathbf{b}_d))^T \left(\sum_{i=1}^N (\mathbf{x}^{(i)} - \mathbf{W}_d \mathbf{W}_e \mathbf{x}^{(i)} - (\bar{\mathbf{x}} - \mathbf{W}_d \mathbf{W}_e \bar{\mathbf{x}})) \right) \\
&= ((\bar{\mathbf{x}} - \mathbf{W}_d \mathbf{W}_e \bar{\mathbf{x}}) - (\mathbf{W}_d \mathbf{b}_e + \mathbf{b}_d))^T \mathbf{0} \\
&= 0
\end{aligned}$$

$$\begin{aligned}
&\sum_{i=1}^N (c_i - b_i)^T b_i \\
&= \sum_{i=1}^N ((\mathbf{P} - \mathbf{W}_d \mathbf{W}_e)(\mathbf{x}^{(i)} - \bar{\mathbf{x}}))^T ((I - \mathbf{P})(\mathbf{x}^{(i)} - \bar{\mathbf{x}})) \\
&= \sum_{i=1}^N (\mathbf{x}^{(i)} - \bar{\mathbf{x}})^T ((I - \mathbf{P})^T (\mathbf{P} - \mathbf{W}_d \mathbf{W}_e))^T (\mathbf{x}^{(i)} - \bar{\mathbf{x}}) \\
&= \sum_{i=1}^N (\mathbf{x}^{(i)} - \bar{\mathbf{x}})^T \mathbf{0}^T (\mathbf{x}^{(i)} - \bar{\mathbf{x}}) \\
&= 0
\end{aligned}$$

Therefore,

$$\begin{aligned}
&\left(\sum_{i=1}^N \|a_i\|^2 \right) - \left(\sum_{i=1}^N \|b_i\|^2 \right) \\
&= \left(\sum_{i=1}^N \|a_i - c_i\|^2 \right) + \left(\sum_{i=1}^N \|c_i - b_i\|^2 \right) + 2 \left(\sum_{i=1}^N (a_i - c_i)^T c_i \right) + 2 \left(\sum_{i=1}^N (c_i - b_i)^T b_i \right) \\
&= \left(\sum_{i=1}^N \|a_i - c_i\|^2 \right) + \left(\sum_{i=1}^N \|c_i - b_i\|^2 \right) \\
&\geq 0
\end{aligned}$$

□

Define

$$A = \left\{ r \in \mathbb{R} : \exists \mathbf{W}_e, \mathbf{b}_e, \mathbf{W}_d, \mathbf{b}_d, r = \sum_{i=1}^N \|\mathbf{x}^{(i)} - \mathbf{W}_d \mathbf{W}_e \mathbf{x}^{(i)} - (\mathbf{W}_d \mathbf{b}_e + \mathbf{b}_d)\|^2 \right\}$$

$$B = \left\{ r \in \mathbb{R} : \exists \mathbf{P}, r = \sum_{i=1}^N \|\mathbf{x}^{(i)} - \mathbf{P} \mathbf{x}^{(i)} - (\bar{\mathbf{x}} - \mathbf{P} \bar{\mathbf{x}})\|^2 \right\}$$

where $\mathbf{W}_e, \mathbf{b}_e, \mathbf{W}_d, \mathbf{b}_d$ are in the corresponding sets and $\mathbf{P}$ is in the set of n by n projection matrices with $\text{rank} \leq m$.

A is the range of the loss function of deep autoencoder and B is the range of the loss function of PCA.

Theorem B.2. *If A and B are defined as above,*

$$B \subset A$$

Proof.

$$\forall b \in B, \exists \mathbf{P}, b = \sum_{i=1}^N \|\mathbf{x}^{(i)} - \mathbf{P} \mathbf{x}^{(i)} - (\bar{\mathbf{x}} - \mathbf{P} \bar{\mathbf{x}})\|^2$$

Define arbitrarily $b_0 \in B$.

Define $\mathbf{P}_0$ as a value of $\mathbf{P}$ for $b = b_0$.

Therefore, $b_0 = \sum_{i=1}^N \|\mathbf{x}^{(i)} - \mathbf{P}_0 \mathbf{x}^{(i)} - (\bar{\mathbf{x}} - \mathbf{P}_0 \bar{\mathbf{x}})\|^2$.

Define arbitrarily $v_1, \dots, v_{\text{rank}(\mathbf{P}_0)}$ to be a basis of the column space of $\mathbf{P}_0$.

Define $\mathbf{V}$ in the set of n by m matrices such that $\mathbf{V} = (v_1 | \dots | v_{\text{rank}(\mathbf{P}_0)} | 0 | \dots | 0)$. Notice that $\text{rank}(\mathbf{P}_0) \leq m$.

Define $\mathbf{W}_{e,0} = \mathbf{V}^T$, $\mathbf{b}_{e,0} = -\mathbf{V}^T \bar{\mathbf{x}}$, $\mathbf{W}_{d,0} = \mathbf{V}$, $\mathbf{b}_{d,0} = \bar{\mathbf{x}}$.

Therefore, $\mathbf{W}_{d,0} \mathbf{W}_{e,0} = \mathbf{P}_0$ and $\mathbf{W}_{d,0} \mathbf{b}_{e,0} + \mathbf{b}_{d,0} = \bar{\mathbf{x}} - \mathbf{P}_0 \bar{\mathbf{x}}$.

Therefore, $b_0 = \sum_{i=1}^N \|\mathbf{x}^{(i)} - \mathbf{W}_{d,0} \mathbf{W}_{e,0} \mathbf{x}^{(i)} - (\mathbf{W}_{d,0} \mathbf{b}_{e,0} + \mathbf{b}_{d,0})\|^2$.

Therefore, $b_0 \in A$. □

A and B are non-empty, bounded below by 0 and closed.

Therefore, A and B each have a minimum.

Therefore, $\min A = \min B$.

References

- [1] C. Sorzano, J. Vargas, and A. Pascual-Montano. A survey of dimensionality reduction techniques. <https://arxiv.org/pdf/1403.2877>, 2014. Accessed: 15 June 2025.
- [2] Z. Jaadi. A step by step explanation of principal component analysis. <https://builtin.com/data-science/step-step-explanation-principal-component-analysis>, 2024. Built In. Accessed: 15 June 2025.

- [3] I. Jolliffe. *Principal Component Analysis*. Springer, New York, 2002.
- [4] I. Jolliffe and J. Cadima. Principal component analysis: A review and recent developments. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 374(2065):20150202, 2016.
- [5] X. Li and U. Davis. Sta135 lecture 2: Sample mean vector and sample covariance matrix. https://www.stat.ucdavis.edu/~xdgli/Xiaodong_Li_Teaching_files/135Note2.pdf. Accessed: 15 June 2025.
- [6] L. Smith. A tutorial on principal components analysis chapter 1. https://www.iro.umontreal.ca/~pift6080/H09/documents/papers/pca_tutorial.pdf, 2002. Accessed: 15 June 2025.
- [7] G. Tanner. Kernel pca. <https://ml-explained.com/blog/kernel-pca-explained>. Machine Learning Explained. Accessed: 15 June 2025.
- [8] Q. Wang. Kernel principal component analysis and its applications in face recognition and active shape models. <https://arxiv.org/pdf/1207.3538>. Accessed: 15 June 2025.
- [9] B. Schölkopf, A. Smola, and K.-R. Müller. Kernel principal component analysis. https://graphics.stanford.edu/courses/cs233-25-spring/ReferencedPapers/scholkopf_kernel.pdf. Accessed: 15 June 2025.
- [10] M. P. Deisenroth, A. A. Faisal, and C. S. Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020. Accessed: 15 June 2025.
- [11] B. Lique, S. Moka, and Y. Nazarathy. *Mathematical Engineering of Deep Learning*. Chapman & Hall/CRC Data Science Series, 2023. Accessed: 15 June 2025.
- [12] E. Plaut. From principal subspaces to principal components with linear autoencoders. *arXiv preprint arXiv:1804.10253*, 2018. Accessed: 15 June 2025.
- [13] R. Grosse. Csc321 lecture 20: Autoencoders. https://www.cs.toronto.edu/~rgrosse/courses/csc321_2017/slides/lec20.pdf, 2017. Accessed: 15 June 2025.
- [14] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift, 2015.
- [15] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization, 2016.
- [16] D. Ulyanov, A. Vedaldi, and V. Lempitsky. Instance normalization: The missing ingredient for fast stylization, 2017.
- [17] Y. Wu and K. He. Group normalization, 2018.
- [18] W. B. Johnson and J. Lindenstrauss. Extensions of lipschitz mappings into a hilbert space. <https://stanford.edu/class/cs114/readings/JL-Johnson.pdf>, 1984.
- [19] B. Ghojogh, A. Ghodsi, F. Karayay, and M. Crowley. Locally linear embedding and its variants: Tutorial and survey. <https://arxiv.org/abs/2011.10925>, 2020. Accessed: 15 June 2025.
- [20] GeeksforGeeks. Curse of dimensionality in machine learning. <https://www.geeksforgeeks.org/machine-learning/curse-of-dimensionality-in-machine-learning/>, 2024. Accessed: 1 June 2025.

- [21] A. Mahajan. Linear vs. non-linear dimensionality reduction: Pca and kernel-pca. <https://medium.com/@abhishek8694/linear-vs-non-linear-dimensionality-reduction-pca-and-kernel-pca-10490f345ba9>, 2023. Accessed: 15 June 2025.
- [22] J. Murel and E. Kavlakoglu. Dimensionality reduction. <https://www.ibm.com/think/topics/dimensionality-reduction>, 2024. IBM.com. Accessed: 15 June 2025.
- [23] Encord. Top 12 dimensionality reduction techniques for machine learning. <https://encord.com/blog/dimensionality-reduction-techniques-machine-learning/>, 2024. Accessed: 15 June 2025.
- [24] P. Geelen. Dimensionality reduction and the advantage of non-linear, reproducible methods like umap. <https://medium.com/aimonks/dimensionality-reduction-and-the-advantage-of-non-linear-reproducible-methods-like-umap>, 2023. Accessed: 15 June 2025.
- [25] G. Lawton. Gans vs. vaes: What is the best generative ai approach? <https://www.techtarget.com/searchenterpriseai/feature/GANs-vs-VAEs-What-is-the-best-generative-AI-approach>, 2023. TechTarget. Accessed: 15 June 2025.